

Direct Line Algorithm

```
#include <graphics.h>
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int gd = DETECT, gm;
    initwindow(800, 600, (char*)"Direct Line Drawing Algorithm");

    int x1, y1, x2, y2;
    cout << "Enter line coordinates (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    float dx = x2 - x1;
    float dy = y2 - y1;

    if (dx == 0) {
        for (int y = min(y1, y2); y <= max(y1, y2); y++) {
            putpixel(x1, y, WHITE);
        }
    } else {
        float m = dy / dx;
        if (abs(m) <= 1) {
            for (int x = min(x1, x2); x <= max(x1, x2); x++) {
                float y = y1 + m * (x - x1);
                putpixel(x, round(y), WHITE);
            }
        } else {
```

```

        // Iterate over y
        for (int y = min(y1, y2); y <= max(y1, y2); y++) {
            float x = x1 + (y - y1) / m;
            putpixel(round(x), y, WHITE);
        }
    }
}

outtextxy(50, 30, (char*)"Direct Line Algorithm");
getch();
closegraph();
return 0;
}

```

DDA Differential Algorithm

```

#include <graphics.h>
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int gd = DETECT, gm;
    initwindow(800, 600, (char*)"DDA Line Drawing");

    int x1, y1, x2, y2;
    cout << "Enter line coordinates (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    int dx = x2 - x1;
    int dy = y2 - y1;

```

```

int steps = max(abs(dx), abs(dy));

float Xinc = dx / (float)steps;
float Yinc = dy / (float)steps;

float x = x1, y = y1;
for (int i = 0; i <= steps; i++) {
    putpixel(round(x), round(y), WHITE);
    x += Xinc;
    y += Yinc;
}

outtextxy(50, 30, (char*)"DDA Line");
getch();
closegraph();
}

```

Bresenham's Line Algorithm

```

#include <graphics.h>
#include <iostream>
using namespace std;

int main() {
    int gd = DETECT, gm;
    initwindow(800, 600, (char*)"Bresenham Line Drawing");

    int x1, y1, x2, y2;
    cout << "Enter line coordinates (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;
}

```

```

int dx = abs(x2 - x1), dy = abs(y2 - y1);
int sx = (x1 < x2) ? 1 : -1;
int sy = (y1 < y2) ? 1 : -1;
int err = dx - dy;

while (true) {
    putpixel(x1, y1, WHITE);
    if (x1 == x2 && y1 == y2) break;
    int e2 = 2 * err;
    if (e2 > -dy) { err -= dy; x1 += sx; }
    if (e2 < dx) { err += dx; y1 += sy; }
}

outtextxy(50, 30, (char*)"Bresenham Line");
getch();
closegraph();
}

```

Bresenham's Circle Algorithm

```

#include <graphics.h>
#include <iostream>
using namespace std;

void drawCirclePoints(int xc, int yc, int x, int y) {
    putpixel(xc+x, yc+y, WHITE);
    putpixel(xc-x, yc+y, WHITE);
    putpixel(xc+x, yc-y, WHITE);
    putpixel(xc-x, yc-y, WHITE);
}

```

```

    putpixel(xc+y, yc+x, WHITE);
    putpixel(xc-y, yc+x, WHITE);
    putpixel(xc+y, yc-x, WHITE);
    putpixel(xc-y, yc-x, WHITE);
}

```

```

int main() {
    int gd = DETECT, gm;
    initwindow(800, 600, (char*)"Bresenham Circle Drawing");

```

```

    int xc, yc, r;
    cout << "Enter center (xc yc) and radius r: ";
    cin >> xc >> yc >> r;

```

```

    int x = 0, y = r;
    int p = 3 - 2*r;

```

```

    while (x <= y) {
        drawCirclePoints(xc, yc, x, y);
        if (p < 0) {
            p += 4*x + 6;
        } else {
            p += 4*(x - y) + 10;
            y--;
        }
        x++;
    }

```

```

    outtextxy(50, 30, (char*)"Bresenham Circle");
    getch();

```

```
    closegraph();  
}
```

Midpoint Circle Algorithm

```
#include <graphics.h>  
  
#include <iostream>  
  
#include <cmath>  
  
using namespace std;  
  
void drawCirclePoints(int xc, int yc, int x, int y) {  
    putpixel(xc+x, yc+y, WHITE);  
    putpixel(xc-x, yc+y, WHITE);  
    putpixel(xc+x, yc-y, WHITE);  
    putpixel(xc-x, yc-y, WHITE);  
    putpixel(xc+y, yc+x, WHITE);  
    putpixel(xc-y, yc+x, WHITE);  
    putpixel(xc+y, yc-x, WHITE);  
    putpixel(xc-y, yc-x, WHITE);  
}  
  
int main() {  
    int gd = DETECT, gm;  
    initwindow(800, 600, (char*)"Midpoint Circle Drawing");  
  
    int xc, yc, r;  
    cout << "Enter center (xc yc) and radius r: ";
```

```

cin >> xc >> yc >> r;

int x = 0, y = r;
int p = 1 - r;

while (x <= y) {
    drawCirclePoints(xc, yc, x, y);
    x++;
    if (p < 0) {
        p += 2*x + 1;
    } else {
        y--;
        p += 2*(x - y) + 1;
    }
}

outtextxy(50, 30, (char*)"Midpoint Circle");
getch();
closegraph();
}

```

2d Transformation

```

#include <graphics.h>
#include <cmath>
#include <iostream>

using namespace std;

int tri[3][2];

```

```
void drawTriangle() {  
    line(tri[0][0], tri[0][1], tri[1][0], tri[1][1]);  
    line(tri[1][0], tri[1][1], tri[2][0], tri[2][1]);  
    line(tri[2][0], tri[2][1], tri[0][0], tri[0][1]);  
}
```

```
void translate2D(int tx, int ty) {  
    for(int i = 0; i < 3; i++) {  
        tri[i][0] += tx;  
        tri[i][1] += ty;  
    }  
}
```

```
void scale2D(float sx, float sy) {  
    int cx = (tri[0][0] + tri[1][0] + tri[2][0]) / 3;  
    int cy = (tri[0][1] + tri[1][1] + tri[2][1]) / 3;  
  
    for(int i = 0; i < 3; i++) {  
        int x = tri[i][0] - cx;  
        int y = tri[i][1] - cy;  
        tri[i][0] = cx + round(x * sx);  
        tri[i][1] = cy + round(y * sy);  
    }  
}
```

```
void rotate2D(float theta) {  
    int cx = (tri[0][0] + tri[1][0] + tri[2][0]) / 3;  
    int cy = (tri[0][1] + tri[1][1] + tri[2][1]) / 3;
```

```
for(int i = 0; i < 3; i++) {  
    int x = tri[i][0] - cx;  
    int y = tri[i][1] - cy;  
  
    tri[i][0] = cx + round(x * cos(theta) - y * sin(theta));  
    tri[i][1] = cy + round(x * sin(theta) + y * cos(theta));  
}  
}
```

```
int main() {  
    cout << "Enter coordinates for Point 1 (x y): ";  
    cin >> tri[0][0] >> tri[0][1];  
  
    cout << "Enter coordinates for Point 2 (x y): ";  
    cin >> tri[1][0] >> tri[1][1];  
  
    cout << "Enter coordinates for Point 3 (x y): ";  
    cin >> tri[2][0] >> tri[2][1];  
  
    int tx, ty;  
    float sx, sy, angle;  
  
    cout << "Enter Translation (tx ty): ";  
    cin >> tx >> ty;  
  
    cout << "Enter Scaling factors (sx sy): ";  
    cin >> sx >> sy;  
  
    cout << "Enter Rotation angle (in radians): ";  
    cin >> angle;
```

```
int gd = DETECT, gm;

initwindow(800, 600, (char*)"2D Transformation");


drawTriangle();
outtextxy(50, 30, (char*)"Original Triangle");
getch();
cleardevice();


translate2D(tx, ty);
drawTriangle();
outtextxy(50, 30, (char*)"After Translation");
getch();
cleardevice();


scale2D(sx, sy);
drawTriangle();
outtextxy(50, 30, (char*)"After Scaling");
getch();
cleardevice();


rotate2D(angle);
drawTriangle();
outtextxy(50, 30, (char*)"After Rotation");
getch();


closegraph();
return 0;
}
```